

What is version control system

Version control also known as source control is tracking and managing software code changes

Famous VCS:

Git (most popular)

Intro of GIT

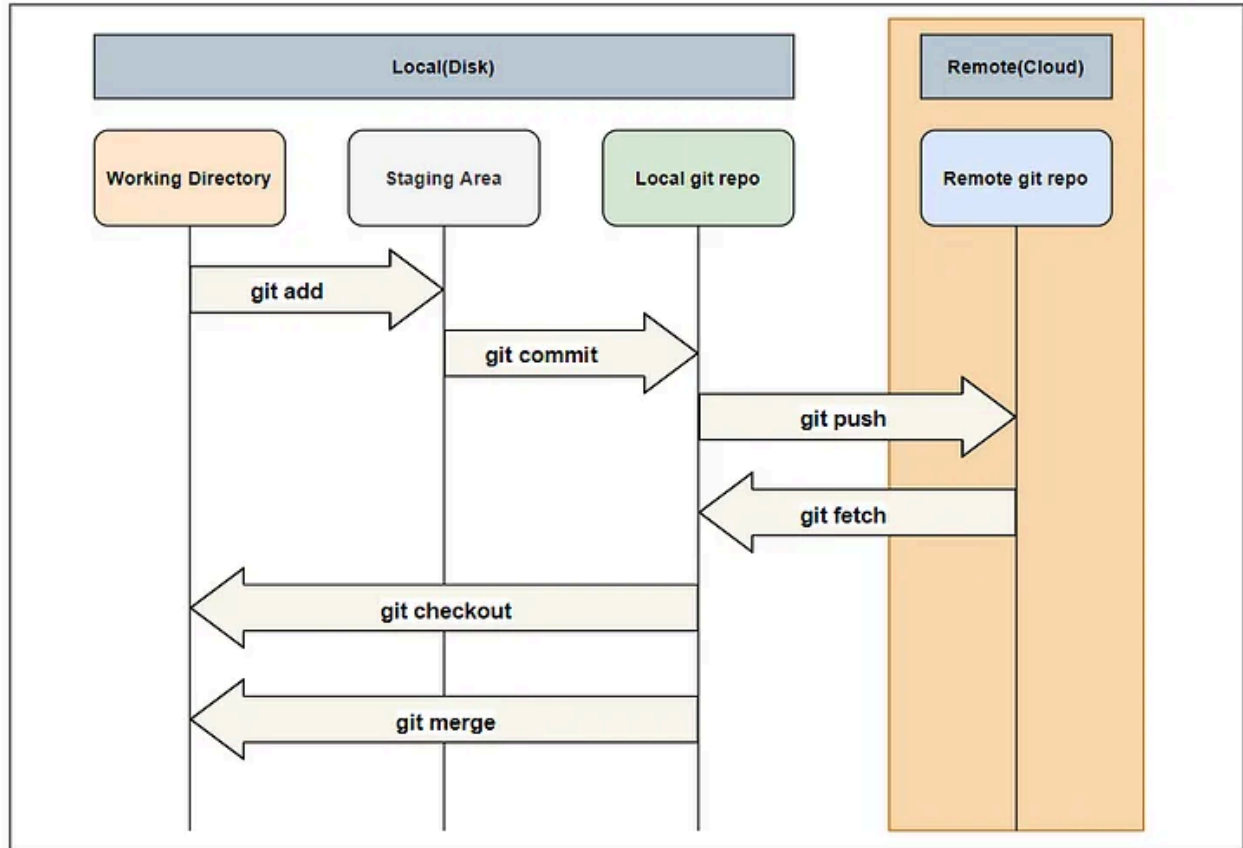
What is Git ?

Git is a free and open-source version control system(tool). Unlike older centralized version control systems such as SVN and CVS

Git also has excellent support for branching, merging, and rewriting repository history, which has led to many innovative and powerful workflows and tools.

Pull requests are one such popular tool that allows teams to collaborate on Git branches and efficiently review each other's code and deliver code independently.

Basic Flow of Commands Between Local and Remote Repositories:



Let's start with basics step by step >

Basics

1 > Git installation

`Apt install git -y`

Or

`Yum install git -y`

2 > Setting Up Git

Once Git is installed, the first step is to set your username and email, which will be attached to your commits replacing "Your Name" with your actual name and "your.email@domain.com" with your email address

`git config --global user.name "Your Name"`

`git config --global user.email "your.email@domain.com"`

3 > Your First Repository

To start using Git, you first need to initialize a repository, which is a designated folder that Git will track for version control

`git init`

4 > Add files (adding file to staging area)

The first command you'll need is `git add`, which adds files to the staging area to be committed. For example, to add a file named `index.html`, you would run the following command

```
git add index.html
```

5 > git commit

After adding the file to the staging area, you can make a commit using the `git commit` command. A commit is a record of the changes you have made to the repository. To commit the staged files with a meaningful message

```
git commit -m "Add initial homepage"
```

6 > check the status

use the `git status` command to check the status of your repository, including which files are staged, unstaged, or untracked

```
git status
```

7 > Git log

This command displays the entire commit history using the default format. You can use this command with the customized options to see different format output.

```
git log
```

8 > git diff

Show the difference between your index(staging area) and the working directory

`git diff`

9 > Restore changes form staging area

Use this command to un-stage changes from the staging area after adding them using the git add command. This command is helpful when you want to change/update a file that you have just staged.

`git restore --staged <file>`

10 > Restore changes form staging area

Show a log of changes to the local repository's HEAD. Add --relative-date flag to show date info or --all to show all refs

```
vikram@LAPTOP-7LSJ4VEF MINGW64 /c/myLocalGitRepo (master)
$ git reflog
db9e7c0 (HEAD -> master, myLocalDevBranch) HEAD@{0}: checkout: moving from master to master
db9e7c0 (HEAD -> master, myLocalDevBranch) HEAD@{1}: checkout: moving from myLocalDevBranch to master
db9e7c0 (HEAD -> master, myLocalDevBranch) HEAD@{2}: checkout: moving from master to myLocalDevBranch
db9e7c0 (HEAD -> master, myLocalDevBranch) HEAD@{3}: rebase (finish): returning to refs/heads/master
db9e7c0 (HEAD -> master, myLocalDevBranch) HEAD@{4}: rebase (start): checkout myLocalDevBranch
181a8ed HEAD@{5}: checkout: moving from myLocalDevBranch to master
db9e7c0 (HEAD -> master, myLocalDevBranch) HEAD@{6}: commit: committing test2 file
181a8ed HEAD@{7}: checkout: moving from master to myLocalDevBranch
181a8ed HEAD@{8}: commit (amend): second commit after first commit in test.txt
bff49b5 HEAD@{9}: reset: moving to HEAD~1
9cbf83f HEAD@{10}: reset: moving to HEAD
9cbf83f HEAD@{11}: commit: new commit after adding second line in test.txt
bff49b5 HEAD@{12}: commit (initial): first commit in test.txt
```

11 > Revert the changes

`git revert <commit>`: Create a new commit that undoes all of the changes made in <commit>, then apply it to the current branch

```
git revert <commit>
```

12 > Remove <file> from the staging area

Remove <file> from the staging area, but leave the working directory unchanged. This un-stages a file without overwriting any changes

```
git reset <file>
```

13 > Create a branch

One of Git's most powerful features is branching, which allows you to experiment with different versions of a project by diverging from the main line of development. The main branch is conventionally called main, although it was previously referred to as master.

To create a new branch, use the `git branch` command followed by the desired branch name. For example, to create a branch named new-feature, run the following command

```
git branch new-feature
```

14 > Changing branch

Once you have created a branch, you can start working on it by switching to the branch using the `git checkout` command

```
git checkout new-feature
```

15 > Merge branch

Any commits you make on this branch will be separate from the main branch and won't affect it. This allows you to work on new features or bug fixes without disrupting the stability of the main branch.

When your feature is complete, you can merge the changes back into the main branch. To do this, switch to the main branch using `git checkout` and then run the following command

```
git merge new-feature
```

16 > to see branch

```
Git branch --list
```


Working with Remote Repositories on GitHub

While Git allows you to work with local repositories on your machine, collaborating and sharing code often requires a remote repository hosted on a platform like GitHub. To work with a remote repository, you first need to create a new empty repository on GitHub.com

17 > Add repo

```
git remote add origin https://github.com/your-account/repo-name.git
```

```
git remote -v
```

18 > Push your code to the repository

This associates your local repository with the GitHub repository. You can then push your code to GitHub using the git push command

```
git push -u origin main
```

19 > Pull the latest code of the remote repository

To pull the latest code from GitHub, use the following command

```
git pull origin main
```

Forking and Cloning Repositories

Forking and cloning repositories are essential techniques for contributing to open-source projects. Forking allows you to create a copy of a repository under your GitHub account. You can then clone this forked repository to your local machine using the following command

20 > Cloning the repository

```
git clone https://github.com/your-account/forked-repo.git
```

21 > Create a branch

You can create a new branch for your changes using the git checkout -b command

```
git checkout -b improvements
```

22 > push the changes

After making the desired changes, commit them and push the branch to your forked repository

```
git push origin improvements
```